

Sri Lanka Institute of Information Technology



IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Test Answer Submission

Student ID	IT24103921
Student Name	Samarawickrama N. A. N. D
Campus/ Center name	Malabe
Specialization	Software Engineering
Weekday/ Weekend	Weekend
Batch No	Y2.S2.WE.SE.01.001

Question 1:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
int main() {  
    int n, i;  
    pid_t pid;  
  
    printf("Enter a positive integer: ");  
    scanf("%d", &n);  
  
    pid = fork();  
  
    if (pid == 0) {  
        int a = 0, b = 1, next;  
  
        printf("PID : %d\n", getpid());  
        printf("PPID : %d\n", getppid());  
  
        printf("Fibonacci Series: ");  
  
        for (i=0; i<n; i++) {  
            if (i <= 1) {  
                next = i;
```

```
        } else {  
            next = a + b;  
            a = b;  
            b = next;  
        }  
  
        printf("%d ", next);  
    }  
  
    printf("\n");  
  
} else {  
    wait(NULL);  
    printf("\nParent Process : Child has completed execution. \n");  
}  
  
return 0;  
}
```

Question 2:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
void* printPrimeNumbers(void* arg) {
```

```
    int n = *(int*)arg;
```

```
    printf("\n--Worker Thread--\n");
```

```
    printf("Thread ID: %lu\n", pthread_self());
```

```
    printf("Process ID: %d\n", getpid());
```

```
    printf("Prime numbers upto %d :\n", n);
```

```
    for (int i = 2; i <= n; i++) {
```

```
        int count = 0;
```

```
        for (int j = 1; j <= i; j++) {
```

```
            if (i % j == 0) {
```

```
                count++;
```

```
            }
```

```
        }
```

```
        if (count == 2) {
```

```
            printf("%d ", i);
```

```
        }
```

```
    }
```

```
        printf("\n");
        pthread_exit(NULL);
    }

int main () {
    pthread_t thread;
    int n;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    pthread_create(&thread, NULL, printPrimeNumbers, &n);
    pthread_join(thread, NULL);

    printf("\nMain Thread: Worker thread finished execution.\n");

    return 0;
}
```

Question 3:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#define MAX 1000
```

```
int arr[MAX];
```

```
int total_sum = 0;
```

```
typedef struct {
```

```
    int start;
```

```
    int end;
```

```
} ThreadData;
```

```
void* worker(void* arg) {
```

```
    ThreadData* data = (ThreadData*)arg;
```

```
    int partial_sum = 0;
```

```
    int count = 0;
```

```
    for (int i = data->start; i <= data->end; i++) {
```

```
        partial_sum += arr[i];
```

```
        count++;
```

```
    }
```

```
    printf("--Worker Thread--\n");
```

```
    printf("Thread ID: %lu\n", pthread_self());
```

```

        printf("Process ID: %d\n", getpid());

        printf("Assigned Range: %d to %d\n", data->start, data->end);

        printf("Partial Sum: %d\n\n", partial_sum);


        total_sum += partial_sum;


        pthread_exit(NULL);
    }

int main () {

    int n, t;


    printf("Enter size of array: ");
    scanf("%d", &n);


    if (n <= 0 || n > MAX) {
        printf("Invalid size of array !\n");
        return 1;
    }


    printf("Enter %d integers: \n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }


    printf("Enter number of threads (<=%d)\n", n);

```

```

scanf("%d", &t);

if (t <= 0 || t > n) {
printf("Invalid threads count !\n");
return 1;
}

pthread_t threads[t];
ThreadData data[t];

int chunk = n / t;
int remainder = n % t;
int start = 0;

for (int i = 0; i < t; i++) {
    data[i].start = start;
    data[i].end = start + chunk - 1;

    if (remainder > 0) {
        data[i].end++;
        remainder--;
    }

    start = data[i].end + 1;

    pthread_create(&threads[i], NULL, worker, &data[i]);
}

```



```
    for (int i = 0; i < t; i++) {  
        pthread_join(threads[i], NULL);  
    }  
  
    double average = (double) total_sum / n;  
  
    printf("Total Sum: %d\n", total_sum);  
    printf("Average: %.2f\n", average);  
  
    return 0;  
}
```